

ARCHITECTURE DOCUMENTATION

Bulk Meter Reading Mobile Application

Target Framework: **React Native (Expo SDK 54)** • Status: **Active Development**

This document outlines the architecture, layout, data flows, and design patterns of the **Bulk Meter Reading Mobile Application**. The app is engineered to support utility workers performing bulk electricity/water meter readings, with a core focus on **offline-first functionality**, **type-safety**, and **secure session handling**.

1. CORE TECHNOLOGY STACK

The application is built on top of a modern, reactive, and native-optimized stack:

- **Framework:** React Native (v0.81) via Expo SDK 54 with the New Architecture enabled.
- **Language:** TypeScript (v5.9) for build-time type verification.
- **Routing & Navigation:** Expo Router (v6.0), using file-based routing and folder-grouped structures.
- **Local Offline Database:** SQLite (via `expo-sqlite`), supporting relational tables for readings, bills, and customers.
- **Secure Cache & Sessions:** Expo SecureStore (`expo-secure-store`) for encrypted, secure device storage of tokens and login user metadata.
- **Network Client:** Axios client configured with timeout and custom headers for backend communication.

2. DIRECTORY & FOLDER LAYOUT

The project structure keeps logic, UI components, types, and persistence separate:

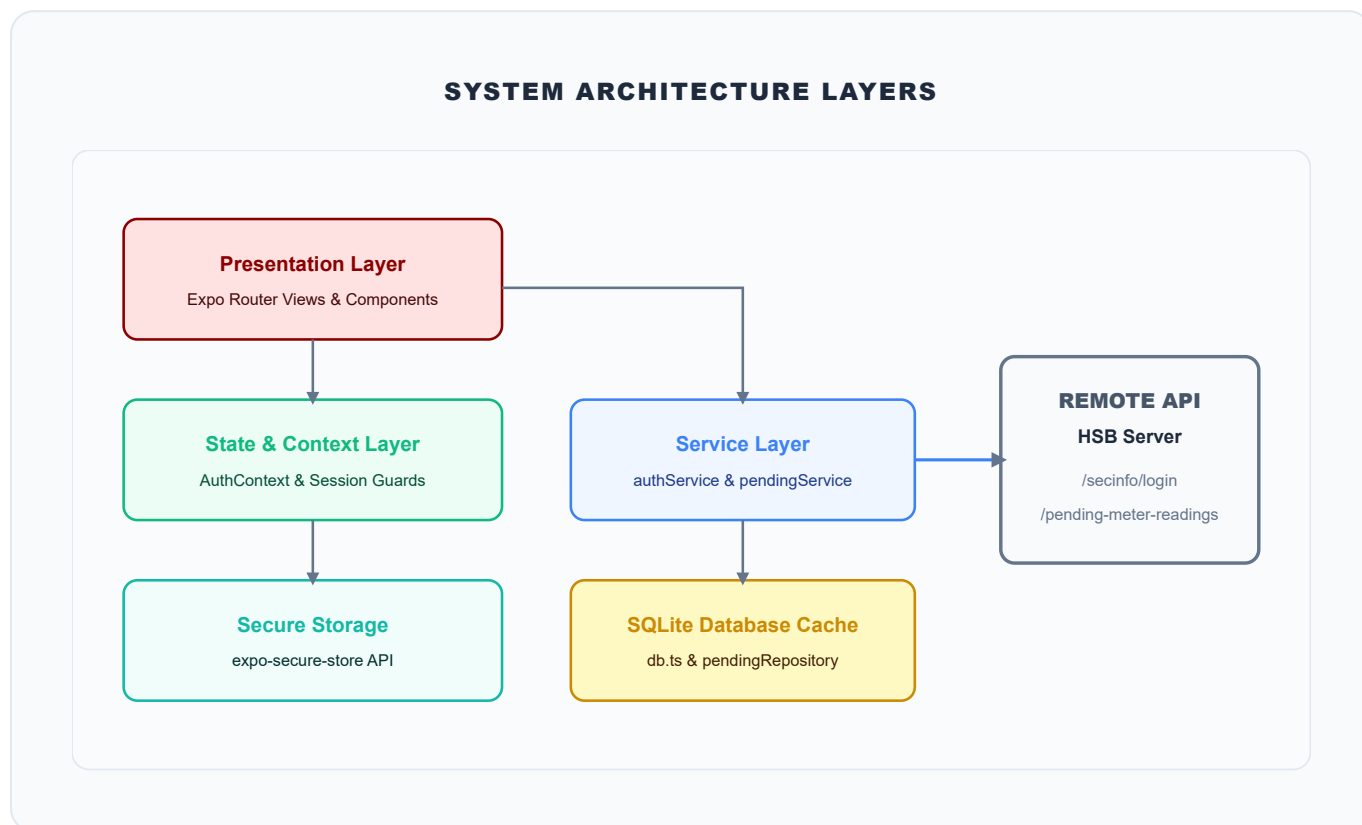
```

mtr-rdng/
├── app/                                # Routing & Presentation Layer (Expo Router)
│   ├── (auth)/                        # Unauthenticated route group
│   │   ├── _layout.tsx               # Layout wrapper for authentication screens
│   │   └── index.tsx                 # Login Screen (Username & Password inputs)
│   ├── (protected)/                  # Authenticated route group (Guarded)
│   │   ├── pending-readings/         # Pending Readings Sub-module
│   │   │   └── index.tsx             # List of customers requiring meter readings
│   │   ├── _layout.tsx               # Route Guard (Checks AuthContext session)
│   │   └── dashboard.tsx             # Dashboard displaying statistics
│   └── _layout.tsx                   # Root layout (Initializes AuthProvider & Stack Navigator)
├── src/                               # Core Codebase Directory
│   ├── api/                          # Network Layer (Axios Endpoint Interfaces)
│   │   ├── apiClient.ts              # Axios client instance configuration
│   │   ├── authApi.ts                # Login API post requests
│   │   └── pendingApi.ts             # Active readings search and retrieval requests
│   ├── assets/                       # Static assets (Images, splash icon, logos)
│   ├── components/                   # Shareable View Components
│   │   └── MeterReadingDashboard.tsx # SVG Donut chart dashboard status widget
│   ├── constants/                   # App-wide constants (URLs, theme, values)
│   ├── context/                      # State Management Contexts
│   │   └── AuthContext.tsx           # Authentication session provider and hook
│   ├── database/                     # Offline Data Storage Layer
│   │   ├── db.ts                     # SQLite connection and database initializer
│   │   └── pendingRepository.ts       # CRUD operations for customer readings
│   ├── services/                     # Business & Orchestration Layer
│   │   ├── authService.ts            # Coordinates login sequence and storage updates
│   │   └── pendingService.ts          # Triggers download logic and mapping
│   ├── storage/                      # Encrypted Key-Value Persistence
│   │   └── secureStore.ts             # Wraps Expo Secure Store actions
│   ├── types/                        # TypeScript interfaces and signatures
│   │   ├── Auth.ts                   # Request/Response credentials types
│   │   ├── PendingReading.ts          # Customer reading model schema
│   │   └── Session.ts                 # Secure state session structure
│   └── utils/                         # Common helper utilities
└── package.json                       # Dependencies configuration

```

3. ARCHITECTURAL LAYERS

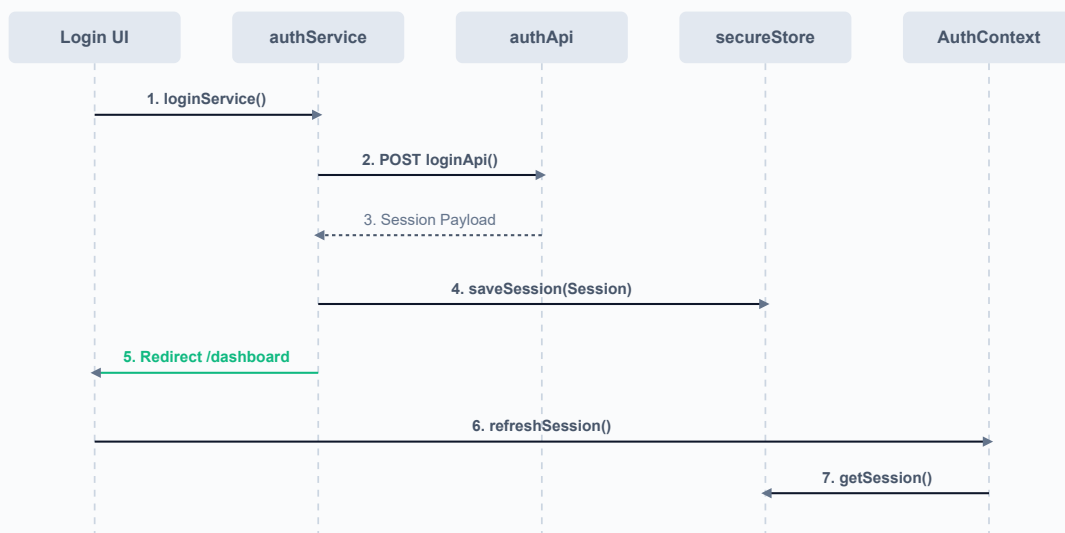
The app adheres to a clean, multi-layered architecture to separate concerns, improve testability, and simplify offline synchronization:



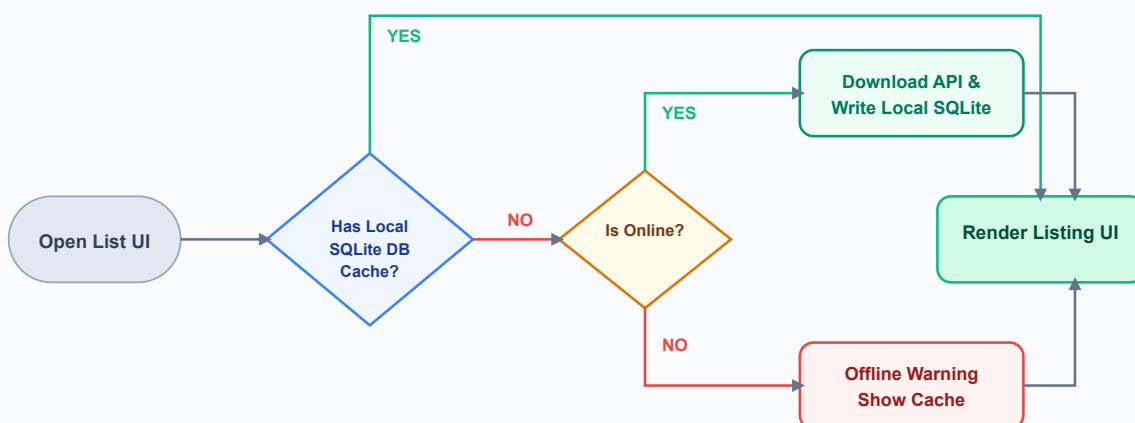
- **Presentation Layer:** Folder groups `(auth)` and `(protected)` segregate guest screens from protected zones. Authentication routing filters access reactive to Context.
- **State & Context Layer:** Managed in `AuthContext.tsx`, allowing reactive, app-wide access to session metadata. Performs non-blocking SecureStore credentials check on cold starts.
- **Orchestration Layer:** `authService.ts` validates inputs, formats requests, and manages the secure credentials lifecycle. `pendingService.ts` handles query coordinates, checking active reader tokens automatically.
- **Data Layer:** Network APIs are structured under `src/api` returning strongly-typed JSON data. SQLite storage caches network inputs onto physical storage via the repository design pattern, preventing logical pollution in UI code.

4. CORE ARCHITECTURAL DATA FLOWS

A. AUTHENTICATION & SECURE STORAGE LOOP



B. OFFLINE-FIRST DOWNLOAD PIPELINE



5. LOCAL SQLITE DATABASE SCHEMA

The SQLite local database acts as the single source of truth for all query listings when the application operates offline.

COLUMN NAME	SQLITE TYPE	CONSTRAINT	DESCRIPTION
installationId	TEXT	PRIMARY KEY	Unique hardware installation ID
accountNumber	TEXT	NOT NULL	Utility customer account identifier
tariff	TEXT	-	Customer tariff billing class
readerCode	TEXT	-	Identifier mapping of the reader
dailyPack	TEXT	-	Identifies sub-route assigned bundles
walkOrder	TEXT	-	Visualized walking sequence for readers
currentBillCycle	TEXT	-	Associated current billing cycle
areaCode	TEXT	-	Assigned geographic operations area
customerCategory	TEXT	-	Billing category (Domestic, Ind, etc)
previousReadingDate	TEXT	-	Date index of previous billing reading
vatApplicable	TEXT	-	Identifies taxation class mappings
currentReading	INTEGER	NULL	Input reading value captured locally
remarks	TEXT	NULL	Capture exceptions or remarks field
syncStatus	TEXT	DEFAULT 'PENDING'	Upload tracker: 'PENDING' or 'SYNCED'